Университет ИТМО
Факультет программной инженерии и компьютерной техники
Дисциплина «Системы компьютерной обработки изображений»
Санкт-Петербург 2026

Группа: P3411
Студент: Горошников Т.И.
Преподаватель: Денисов А.К.

Отчет по лабораторной работе №1 «Реализация базового алгоритма обработки изображения»

Вариант 4: медианный фильтр с переменным размером ядра

Содержание

- Задание
- Ход работы
  - Теоретическое описание метода
  - Описание разработанной системы
  - Результаты работы и тестирования
- Заключение

Задание

1. Реализовать программу согласно варианту задания. Базовый алгоритм, используемый в программе, необходимо реализовать в 2 вариантах: с использованием встроенных функций какой-либо библиотеки (OpenCV, PIL и др.) и нативно на Python или C++.
2. Сравнить быстродействие реализованных вариантов.
3. Сделать отчёт в виде README на GitHub, там же должен быть выложен исходный код.

Ход работы

Теоретическое описание метода

Медианный фильтр заменяет значение пикселя медианой значений в локальном окне. Для окна размера `k x k` в него входит `k^2` значений, а результатом становится элемент с индексом `k^2 // 2` после сортировки.
Фильтр хорошо удаляет импульсный шум и сохраняет резкие границы лучше, чем линейное усреднение. При увеличении размера ядра шум подавляется сильнее, но мелкие детали изображения могут исчезать.
В работе размер ядра задаётся при запуске. Допустимы нечётные значения `3`, `5`, `7` и больше. Для обработки границ используется репликация крайних пикселей (`BORDER_REPLICATE`), что позволяет сохранить размер изображения.

Описание разработанной системы

Ссылка на Github-репозиторий.
Организация файлов и кода:

- `median_filter.py` содержит общий контракт и две реализации:
  - `median_filter_library` использует `cv2.medianBlur`;
  - `median_filter_native` обрабатывает окно циклами Python, сортирует его значения и выбирает медиану.
- `benchmark.py` генерирует `.csv` файлы с результатами сравнения native и library методов.
- `main.py` предоставляет CLI для обработки, сравнения и измерения времени.
- `tests/` содержит тесты параметров, форматов изображений и совпадения результатов.

Результаты работы и тестирования

Сравнение результирующих изображений

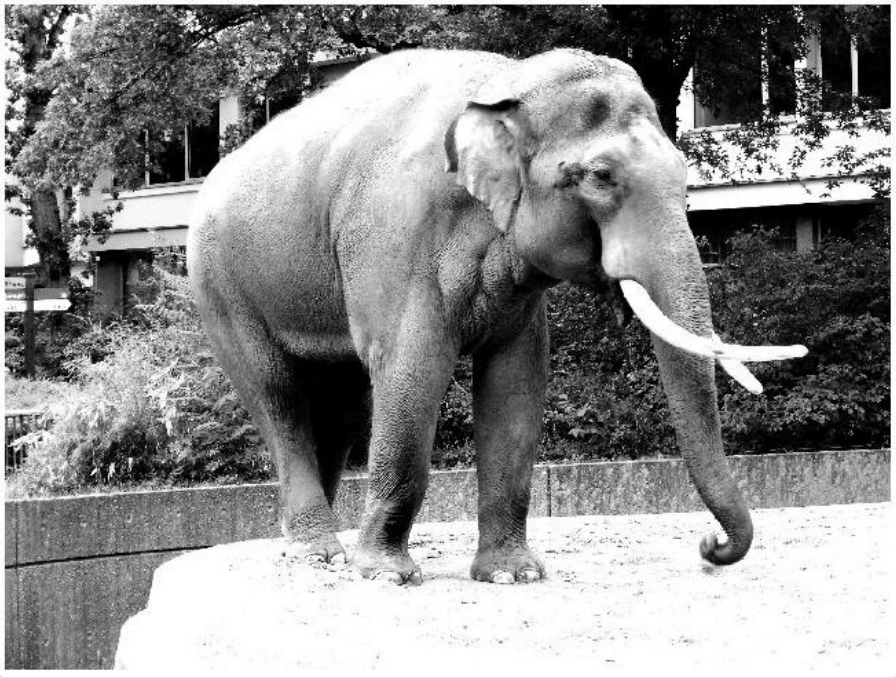
Тестовое изображение 1

Изображение 1, библиотечный метод, ядро 3x3

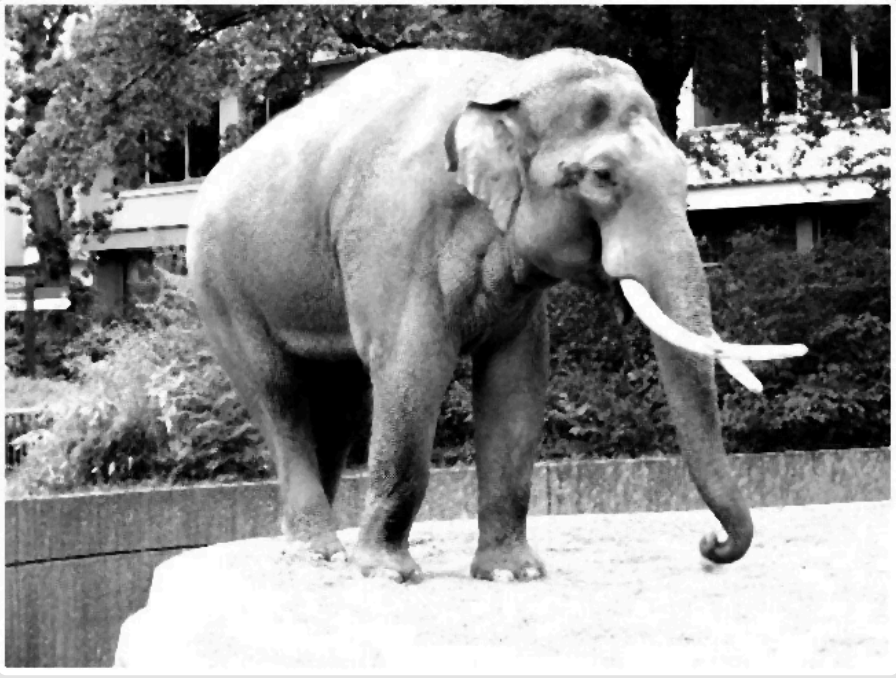
Изображение 1, библиотечный метод, ядро 9x9

Изображение 1, нативный метод, ядро 3x3

Изображение 1, нативный метод, ядро 9x9

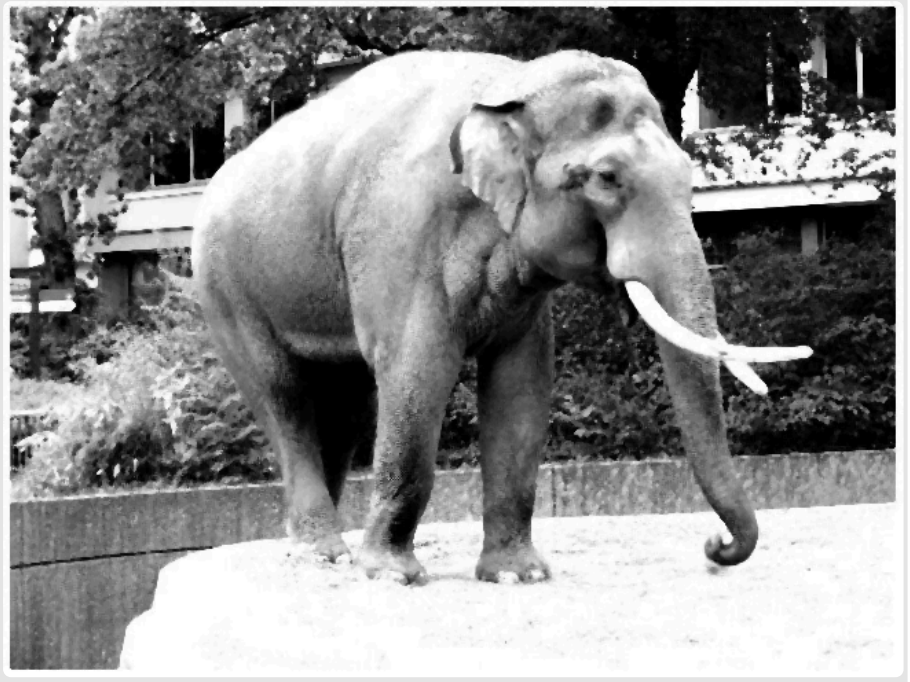


Тестовое изображение 2



Изображение 2, библиотечный метод, ядро 3x3

Изображение 2, библиотечный метод, ядро 9x9



Изображение 2, нативный метод, ядро 3x3

Изображение 2, нативный метод, ядро 9x9

Сравнение метрик

Изображение 1:

Ядро	Метод	Среднее время работы, мс
3x3	OpenCV	1.713
3x3	Native	10622.190
5x5	OpenCV	7.038
5x5	Native	15098.248
7x7	OpenCV	195.463
7x7	Native	23429.213
9x9	OpenCV	206.073
9x9	Native	36491.213

Изображение 2:

Ядро	Метод	Среднее время работы, мс
3x3	OpenCV	0.056
3x3	Native	417.997
5x5	OpenCV	0.360
5x5	Native	586.543
7x7	OpenCV	7.848
7x7	Native	865.119
9x9	OpenCV	8.525
9x9	Native	1324.487

Заключение

В ходе данной лабораторной работы были реализованы два варианта медианного фильтра с переменным нечётным размером ядра: с помощью нативного Python и с помощью библиотеки OpenCV. Результаты тестирования показали, что библиотечная реализация работает значительно быстрее и эффективнее нативной.